COMP5048 Assignment 1A

YAQI MEI

2025.3.26

1 Category A

1.1 Visual Analysis 1

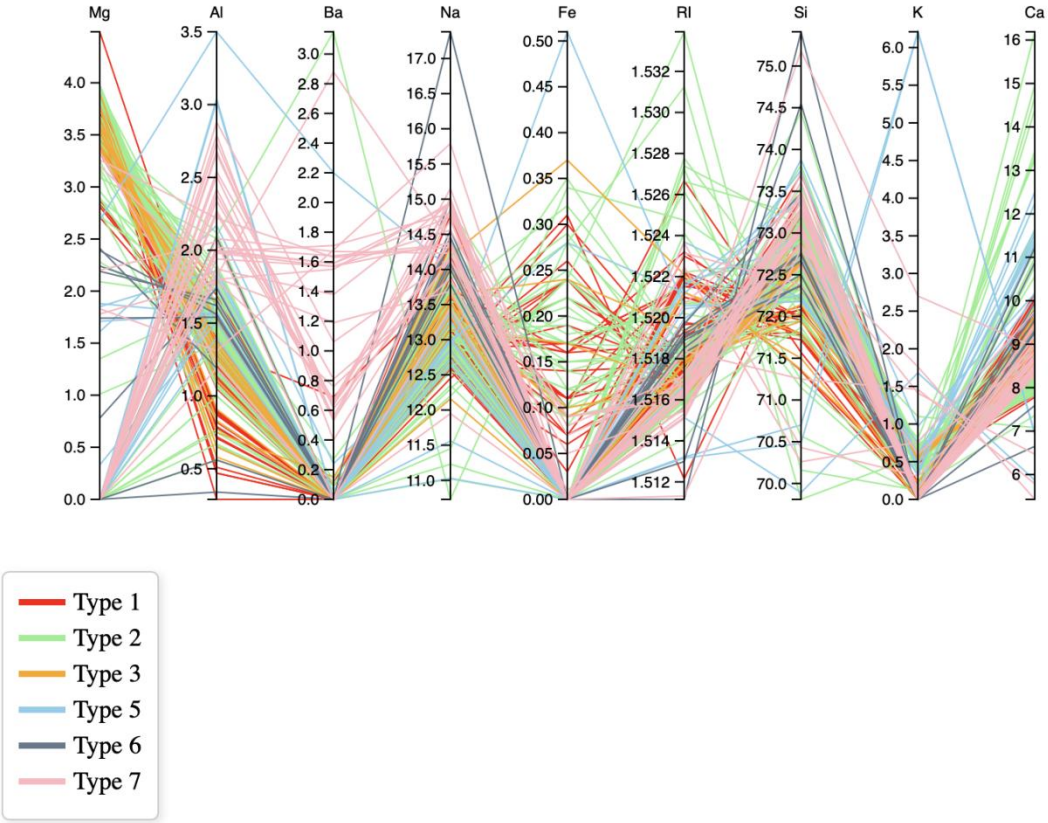

Figure 1: Parallel coordinates

1.2 Discussion on Visual Analysis 1

1.2.1 Design

I focus on two central aspects for visualization design: the choice of tool and the strategy behind axis ordering. Compared with R, D3 was chosen based on aesthetics, analyzability, and interactivity.

After analyzing glass.csv, I believe that the dataset can be divided into feature numbers (9 chemical components) and one categorical variable (Type). Use each line to represent a glass sample, with different colors corresponding to different types of glass.I determine the order based on the correlation between the variables and the target variable (Type, glass type).The current task is to sort the nine feature values. I used the pandas library in Python to calculate the correlation matrix and obtained the order of correlation from high to low.

In addition, we can assist in classification and sorting from the perspective of chemical composition, placing chemical elements with similar properties in similar positions and placing RI as the only physical property in the middle makes observation more convenient.

1.2.2 Evaluation

From the graph, we can intuitively see that type 5 lines do not have significant concentration like other lines, but rather have a dispersed distribution and jumping trend, which may have separability. The lines of type 2 and type 3 overlap highly on the Mg, Al, Na, and RI axes without clear separation patterns, indicating a high possibility of confusion. These types may require additional engineering features or non visual separation strategies. We can also easily see that type 6 and type 7 are almost identical on all axes,both of which indicates similarity.The numerical variation on the Fe axis is significant, almost spanning the entire range of the Fe axis. This may indicate that the influence of Fe content on glass is complex, or that there may be extreme or abnormal values in the data.

As for correlation,taking Na and Si as examples, it can be seen that they exhibit opposite trends in many samples, which may indicate the substitutability between the two stabilizers in glass formulations. I also observed that due to bias in the dataset, there are multiple intersecting lines between the AI and Mg axes that do not show the same upward or downward trend, indicating a negative correlation between them from the graph, which contradicts our common sense. The high correlation between RI and Ca indicates that the refractive index of glass may be affected by the Ca content.It's easy to see the cluster from the graph. take type 7 as an example again,we may see that its lines stay consistent across certain dimensions,indicating that type 7 samples are similar across these features.

1.2.3 Pros and Cons

Pros:

1. Support visualization of multidimensional data,which is very effective for processing high-dimensional data such as glass datasets.
2. Clearly see the differences in trends of various samples on different features, making it easy to identify overlapping values and significantly different values.
3. By observing the synchronous or reverse trends of the sample lines on each axis, potential positive or negative correlation features can be discovered.

Cons:

1. If there are too many samples, the lines are crowded and hard to identify.
2. No quantitative information provided, relying solely on visual perception.This makes it unable to meet tasks that require precise data support in certain situations.
3. Sometimes weak trends may be missed or misled by overlapping lines.
4. Greatly affected by axis order.

1.3 Visual Analysis 2

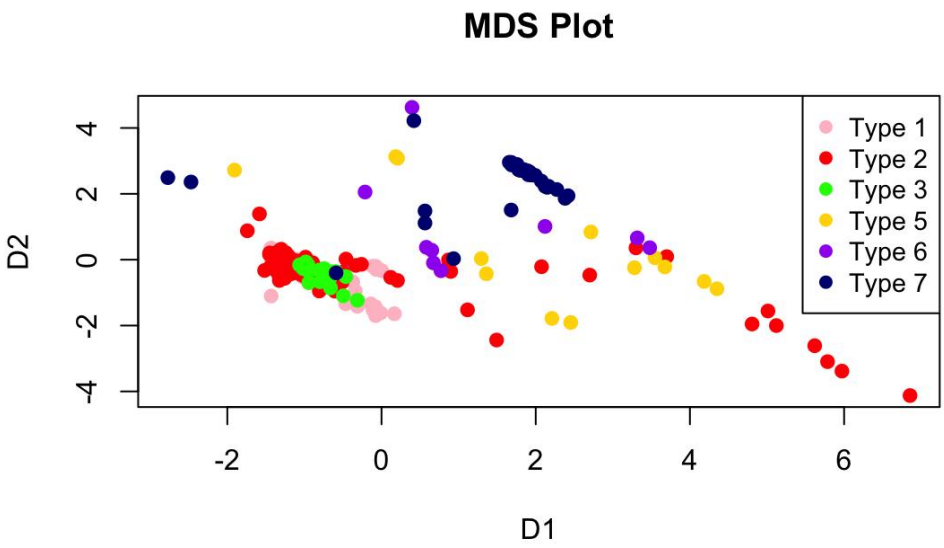

Figure 2: MDS Plot

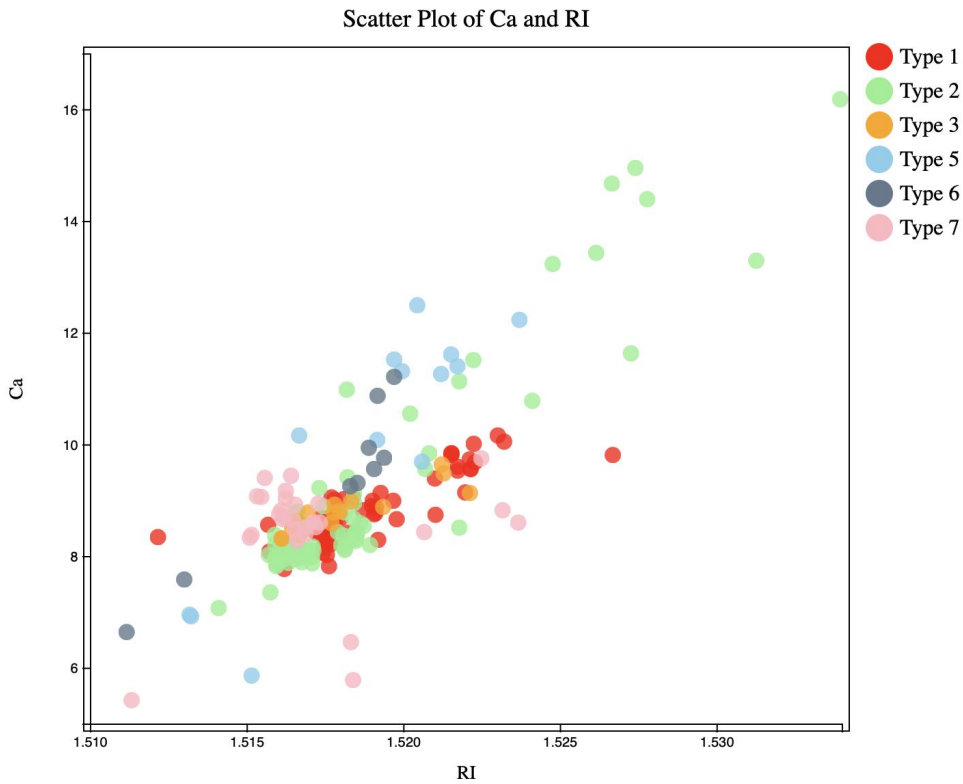

Figure 3: Scatter Plot of Ca and RI

1.4 Discussion on Visual Analysis 2

1.4.1 Design

In terms of tool selection, I used R to generate MDS graphs and distinguished different types with different colors. I also added legends for easy observation. Then, I selected SI and Ca columns and used the D3.js tool to create a scatter plot.

1.4.2 Evaluation

To analyze this MDS plot , we can focus on several aspects:

Firstly, for clusters. Points of the same color which are closer together indicates that these types of glass have similar features in the high-dimensional feature space. We can easily find that type1 and type 3 all have obvious clustering, which also show tightly overlap, suggesting that these two types of glass may be composed of similar chemical compositions. And it is easy to detect type 7 is positioned far apart from the other types, indicating it is very distinct. Secondly is about separation. We can evaluate whether different types are clearly separated. For example, type1 is clustered in the lower left corner of the image, while type 7 is far away and located in the upper right corner. Clear separation indicates that type1 and type 7 are chemically distinct from others. In addition, we can also simply find outliers. In this plot, there are several isolated points in the type 2 and type 7 clusters which may represent nodes with significant differences from other same type nodes.

As for the evaluation of the scatter plot, I have selected two highly correlated features from glass.csv. It is easy to see from the graph that they have a strong positive correlation, and we may conclude that there is indeed a certain correlation between the refractive index of glass and the calcium content. We can also find clusters that are the same as those in the MDS graph, such as type 1, type 2, type 3 and type 7. And for outliers, we can easily detect and draw the same conclusion as the MDS plot.

1.4.3 Pros and Cons

MDS Plot:

Pros: It is useful for high-dimensional datasets.We can intuitively find clusters and outliers through the MDS graph, which makes us easily identify relationships between data points, such as which types of glass are similar or distinct.

Cons: MDS reduces dimensionality, so some information from the original high-dimensional space may be lost. Also, the axes in MDS are abstract, which makes it harder to interpret the plot in terms of specific variables.

Scatter Plot:

Pros: Looks very intuitive and very great for showing the relationship between two variables. Through the graph we can easily find whether there is a linear or non-linear correlation, and the cluster and outliers.

Cons: Only suitble for showing the relationship between two variables and easy to cause information loss.

1.4.4 Comparison

Parallel coordinate graph preserves the actual values of each feature, allowing it to be interpreted based on the original values, and making it easy to identifying patterns across dimensions. But it doesn't seem intuitive to analyze the correlation and clustering of the data. **In contrast**, MDS plots maintain pairwise distances between data points, which helps detect correlation between samples. However, the axes in the MDS graph do not correspond to the original features, thus losing the interpretability of a single dimension. And because scatter plots can only show the relationship between two variables, unlike parallel coordinate plots which can observe multiple variables.

2 Category B

2.1 Visual Analysis 1

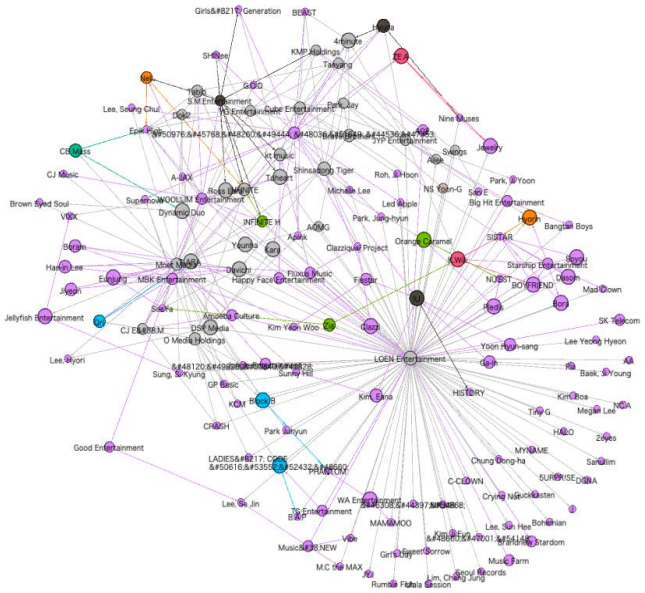

Figure 4: K-pop Top 150 based on Closeness Centrality Visualiaztion

2.2 Discussion on Visual Analysis 2

2.2.1 Design

First using NetworkX for Centrality analysis to generate a new GML files as required, and then I used Gephi to layout and choose Yifan Hu Layout. Drawing size based on closeness centrality and colour based on modularity class.

2.2.2 Evaluation

The nodes are colored differently based on their modularity class, indicating different subgroups in the network. For example, groups of similar colors represent closely related entities.

Closeness centrality measures how close a node is to all other nodes in the network. It can be clearly observed from the graph that the node located in the bottom right corner, which is LOEN Entertainment, becomes the most central node, indicating that its path is very short compared to most other nodes. This indicates that it serves as a bridge between different entertainment companies and artist subgroups, promoting rapid indirect connections. Such companies like MBK Entertainment and JYP Entertainment also show high closeness, making them efficient in reaching others in the network. We can easily conclude that closeness centrality here identifies those high central nodes spread information or influence rapidly. Small nodes near the periphery have lower closeness centrality, which means they are more isolated and take longer to reach other nodes, indicating that the companies or artists have less influence.We can also see some artists at high central nodes, who are highly integrated in the network, which means they may have close collaborative relationships.

Gephi shows the modular value of this graph is 0.471, which means that the cluster effect of this project is significant, which helps identify which companies or artists have greater influence. Applied in Kpop scenarios, it may help decision-makers in selecting cooperation partners.

2.2.3 Pros and Cons

Pros:

Directly identify key nodes and edge nodes. For example, we can easily find the high central node such as LOEN Entertainment.

The different colors and sizes of nodes can intuitively express the strength of centrality, making the network structure clear and intuitive. In K-pop networks, this helps identify which companies or artists have greater potential for dissemination or influence.

We can quickly infer the degree of closeness between artists, companies from this, which is suitable for complex relational networks and may help tthe relevants for decision making.

Cons:

The node position is determined by layout algorithms which may mislead interpretation.

There is also the issue of nodes being too crowded, with many overlapping and poorly readable labels, which seems hard to detect and analyse. So it may require more space or layers to distinguish this information.

At the same time, as a static view, connections between certain nodes might be short-term, yet their relationships in the graph may appear particularly close.

2.3 Visual Analysis 2

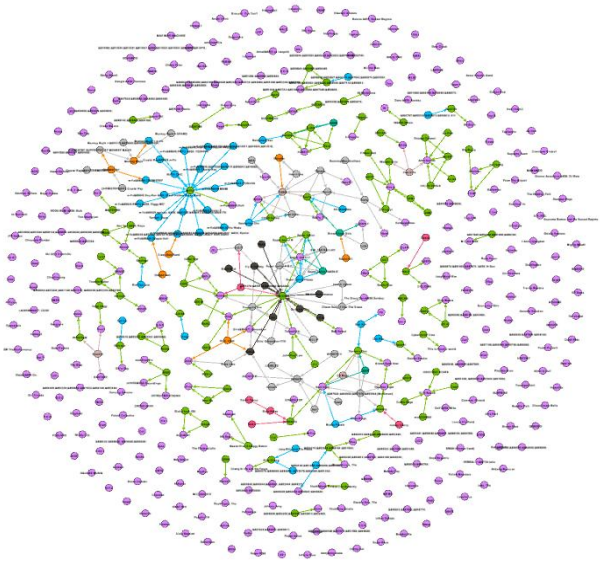

Figure 5: ’group’ Visualization--Fruchterman Reingold layout

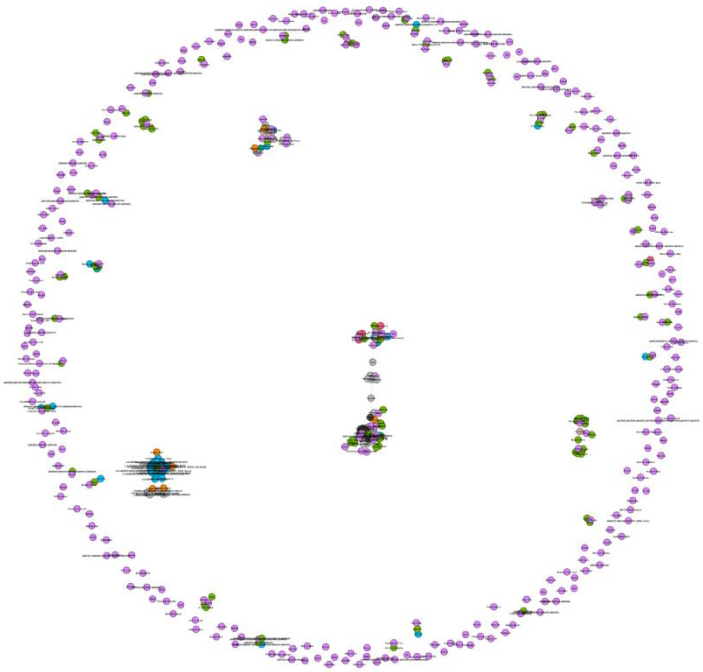

Figure 6: ’group’ Visualization--Yifan Hu (sfdp) layout

2.4 Discussion on Visual Analysis 2

2.4.1 Design

According to the task requirements, I used Python to read the labels in given GML file, and then selected "group" as the research object. I generated all nodes of type "group" and created a new GML file and also chose to use Gephi for visualization, drawing size based on closeness centrality and colour based on modularity class. After that I chose Fruchterman Reingold Layout and Yifan Hu Layout for better analysis.

2.4.2 Evaluation

Same as the previous visualization analysis. The larger nodes at the center of these communities have high closeness centrality, indicating they can quickly reach other nodes. This suggests that these nodes are more efficient in spreading influence or information across the network.

From these two graphs, especially Figure 6, we can clearly see multiple clusters and communities within the artist network labeled as "group". These clusters are formed based on the tight centrality of nodes. Nodes within the same cluster are more closely connected to each other, allowing them to spread information and influence each other faster. From Figure 5, we can clearly see the connections between the large clusters, and the lines between them are more distinct compared to the edge nodes. We can conclude that the internal connections of the large clusters near the center of the circle are more closely interconnected. Regarding outliers, we can see many small nodes distributed around the periphery.

2.4.3 Pros and Cons

Pros:

From this visualization we can clearly and intuitive find cluster and outlier edge values. Targeted screening of research subjects for observation and analysis will make decision-making more efficient. We can also draw the same advantages as the previous task. We can quickly find the central nodes, which means those artists with higher influence and commercial value.

If researchers only want to analyze artists with certain labels, this will also eliminate redundant information and facilitate observation.

Cons:

This view can only highlight one characteristic of the artist's certain label and cannot observe more detailed information about the subject, such as their market performance or fan base. Therefore, the view seems over simplistic and lacks detailed information, which affects deeper analysis.

And when using Yifan Hu layout, the nodes will be visually far away from each other, which may mislead us to think that some nodes are completely isolated.

2.4.4 Comparison

The first visualization method selects the top 150 important nodes from a large sample, so that edge nodes with few connections to other nodes are relatively excluded. We can see more clustering characteristics and quickly select the central node. It may seem like macro level control, which may make decision-making faster when applied in practice, but it also lacks specificity and accuracy.

The second type of visualization observation dataset is smaller and more concentrated, and to some extent has the same characteristics, which will make our research more targeted, purpose oriented, and can effectively avoid cluttered information, making decision-making more reasonable.

Last but not least, two objects show different aims and reseach targets.

3 Appendix : Source Code

3.1 Category A--Visual Analysis 1

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <style>
    #legend {
      display: flex;
      flex-direction: column;
      position: absolute;
      bottom: 230px;
      left: 20px;
      background-color: white;
      border: 1px solid #ccc;
      padding: 10px;
      border-radius: 5px;
      box-shadow: 2px 2px 5px rgba(0, 0, 0, 0.1);
    }

    .legend-item {
      display: flex;
      align-items: center;
      margin-bottom: 5px;
    }

    .legend-color {
      width: 30px;
      height: 4px;
      margin-right: 5px;
    }
  </style>
</head>

<body>
  <!-- Load D3.js -->
  <script src="//d3js.org/d3.v7.min.js"></script>
  <script>
    // set the dimensions and margins of the diagram
    const margin = {top: 50, right: 50, bottom: 50, left: 0},
          width = 900 - margin.left - margin.right,
          height = 400 - margin.top - margin.bottom;
```

```
    // Load data from CSV
    d3.csv("glass.csv").then(function(data) {
      // Create the main SVG container
      const svg = d3.select("body").append("svg")
            .attr("width", width + margin.left + margin.right - 100)
            .attr("height", height + margin.top + margin.bottom)
          .append("g")
            .attr("transform", "translate(" + margin.left + "," + margin.top + ")");

      // Define the dimensions/columns to be visualized
      const dimensions = ["Mg", "Al", "Ba", "Na", "Fe", "RI", "Si", "K", "Ca"];

      // Convert string data to numeric
      data.forEach(d => {
        dimensions.forEach(dim => {
          d[dim] = +d[dim];
        });
      });

      // Create Y scale for each dimension
      const y = {};
      dimensions.forEach(function(dim) {
        y[dim] = d3.scaleLinear()
              .domain(d3.extent(data, d => d[dim]))
              .range([height, 0]);
      });

      // Create X scale for axis placement
      const x = d3.scalePoint()
            .range([0, width - 100])
            .padding(1)
            .domain(dimensions);

      // Function to draw one path for each data row
      function path(d) {
        return d3.line()(dimensions.map(p => [x(p), y[p](d[p])]));
      }

      // Color scale for Type values
      const Color_select = d3.scaleOrdinal()
            .domain(["1", "2", "3", "5", "6", "7"])
            .range(["#FF0000", "#90EE90", "#FFA500", "#87CEEB", "#64788B", "#FFB6C1"]);

      // Draw the polylines
```

```
        svg.selectAll(".myPath")
            .data(data)
            .enter().append("path")
              .attr("d", path)
              .attr("class", "myPath")
              .style("fill", "none")
              .style("stroke", d => Color_select(d.Type))
              .style("opacity", 1);

        // Draw one axis per dimension
        const axes = svg.selectAll(".myAxis")
            .data(dimensions)
            .enter().append("g")
              .attr("transform", d => "translate(" + x(d) + ")")
              .attr("class", "myAxis")
              .each(function(d) { d3.select(this).call(d3.axisLeft().scale(y[d])); });

        // Add axis labels
        axes.append("text")
            .style("text-anchor", "middle")
            .attr("y", -9)
            .text(d => d)
            .style("fill", "black");
    });

    // Legend data for Types
    const legendData = [
      { name: "1", color: "#FF0000" },
      { name: "2", color: "#90EE90" },
      { name: "3", color: "#FFA500" },
      { name: "5", color: "#87CEEB" },
      { name: "6", color: "#64788B" },
      { name: "7", color: "#FFB6C1" }
    ];

    // Create legend container
    const legendContainer = d3.select("body").append("div").attr("id", "legend");

    // Render legend items
    const legendItems = legendContainer.selectAll(".legend-item")
      .data(legendData)
      .enter()
      .append("div")
      .attr("class", "legend-item");
```

```
    // Add colored line block
    legendItems.append("div")
        .style("background-color", d => d.color)
        .attr("class", "legend-color");

    // Add text label
    legendItems.append("div")
        .text(d => "Type " + d.name);
  </script>
</body>
</html>
```

3.2 Category A--Visual Analysis 2

3.2.1 MDS plot

```
Type_color <- function(type){
  if (type == 1){
     return ("pink")
  }
  if (type == 2){
     return ("red")
  }
  if (type == 3){
     return ("green")
  }
  if (type == 4){
     return ("blue")
  }
  if (type == 5){
     return ("gold")
  }
  if (type == 6){
     return ("purple")
  }
  if (type == 7){
     return ("navy")
  }
}

glass = read.csv("glass.csv")

glass_dist <- dist(glass[c(1:9)])

glass_mds <- cmdscale(glass_dist)

colors <- sapply(glass$Type, Type_color)

plot(glass_mds, col = colors, pch = 19, main = "MDS Plot", xlab = "D1", ylab = "D2")

legend('topright',
          c("Type 1", "Type 2", "Type 3", "Type 5", "Type 6", "Type 7"),
          pch = rep(19, 6),
          col = c("pink", "red", "green", "gold", "purple", "navy"),
          cex = 0.9)
```

3.2.2 Scatter plot

```
<!DOCTYPE html>
<meta charset="utf-8">
<script src="https://d3js.org/d3.v7.min.js"></script>

<head>
  <style>
    #legend {
        display: flex;
        flex-direction: column;
        position: absolute;
        top: 50px;
        right: 50px;
    }
    .legend-item {
        display: flex;
        align-items: center;
        margin-bottom: 5px;
    }
    .legend-color {
        width: 20px;
        height: 20px;
        border-radius: 50%;
        margin-right: 5px;
    }
  </style>
</head>

<!-- Containers for the legend and plot -->
<div id="legend"></div>
<div id="glass"></div>

<script>

// Set the dimensions and margins of the graph
var margin = {top: 50, right: 80, bottom: 60, left: 70},
    width = 700 - margin.left - margin.right,
    height = 600 - margin.top - margin.bottom;

// Append the SVG canvas
var svg = d3.select("#glass")
  .append("svg")
    .attr("width", width + margin.left + margin.right)
```

```
      .attr("height", height + margin.top + margin.bottom)
   .append("g")
      .attr("transform", "translate(" + margin.left + "," + margin.top + ")");

// Add a title to the graph
svg.append("text")
      .attr("x", width / 2)
      .attr("y", -20)
      .attr("text-anchor", "middle")
      .style("font-size", "18px")
      .text("Scatter Plot of Ca and RI");

// Load the dataset from CSV file
d3.csv("glass.csv").then(function(data) {

   // Convert data values to numbers
   data.forEach(d => {
      d.RI = +d.RI;
      d.Ca = +d.Ca;
   });

   var x = d3.scaleLinear()
      .domain(d3.extent(data, d => d.RI))
      .range([0, width])
      .nice();

   var y = d3.scaleLinear()
      .domain(d3.extent(data, d => d.Ca))
      .range([height, 0])
      .nice();

   // Add the X-axis
   svg.append("g")
      .attr("transform", "translate(0," + height + ")")
      .call(d3.axisBottom(x).ticks(6));

   // Add the Y-axis
   svg.append("g")
      .call(d3.axisLeft(y).ticks(6));


   svg.append("text")
         .attr("text-anchor", "middle")
         .attr("x", width / 2)
```

```
        .attr("y", height + margin.bottom - 20)
        .style("font-size", "14px")
        .text("RI");

svg.append("text")
        .attr("text-anchor", "middle")
        .attr("transform", "rotate(-90)")
        .attr("y", -margin.left + 20)
        .attr("x", -height / 2)
        .style("font-size", "14px")
        .text("Ca");

var Color_select = d3.scaleOrdinal()
        .domain(["1", "2", "3", "5", "6", "7"])
        .range(["#FF0000", "#90EE90", "#FFA500", "#87CEEB", "#64788B", "#FFB6C1"]);

// Add scatter plot points
svg.append('g')
    .selectAll("dot")
    .data(data)
    .enter()
    .append("circle")
        .attr("cx", d => x(d.RI))
        .attr("cy", d => y(d.Ca))
        .attr("r", 6)
        .style("fill", d => Color_select(d.Type))
        .style("opacity", 0.8);

svg.append("rect")
        .attr("x", -5)
        .attr("y", -5)
        .attr("width", width + 10)
        .attr("height", height + 10)
        .style("fill", "none")
        .style("stroke", "black");

var legendData = [
    { name: "Type 1", color: "#FF0000" },
    { name: "Type 2", color: "#90EE90" },
    { name: "Type 3", color: "#FFA500" },
    { name: "Type 5", color: "#87CEEB" },
    { name: "Type 6", color: "#64788B" },
    { name: "Type 7", color: "#FFB6C1" }
];
```

```
    // Create the legend container
    var legendContainer = d3.select("#legend");
    var legendItems = legendContainer.selectAll(".legend-item")
        .data(legendData)
        .enter()
        .append("div")
        .attr("class", "legend-item");

    legendItems.append("div")
        .style("background-color", d => d.color)
        .attr("class", "legend-color");

    legendItems.append("div")
        .text(d => d.name)
        .attr("class", "legend-text");

});

</script>
```

3.3 Category B--Visual Analysis 1

```
import networkx as nx
import matplotlib.pyplot as plt
G = nx.read_gml('K-pop_lcc_labelled-1.gml')
nx.draw_spring(G, with_labels = False)
cc_G = G
cc = nx.closeness_centrality(cc_G) # CC Process
cc_sort = sorted(cc.items(), key=lambda x:x[1], reverse = True) # Sort
cc_remove = cc_sort[150:] # Select 150 most importment
cc_remove_name = [x[0] for x in cc_remove]
for i in cc_remove_name:
    cc_G.remove_node(i)
nx.draw_spring(cc_G, with_labels = False)
nx.write_gml(cc_G, "K-pop_top150.gml")
```

3.4 Category B--Visual Analysis 2

```
import networkx as nx
import matplotlib.pyplot as plt
G = nx.read_gml('K-pop_lcc_labelled-1.gml')
nx.draw_spring(G)
G.nodes['B.A.P']
name = list()
for i in G.nodes:
    if G.nodes[i]['type'] not in name:
        name.append(G.nodes[i]['type'])
print(name)
G_remove_label = list()
for i in G.nodes:
    if 'group' not in G.nodes[i]['type']:
        G_remove_label.append(i)
for i in G_remove_label:
    G.remove_node(i)
nx.draw_spring(G)
nx.write_gml(G, "lable.gml")
```